

TDDI16: Datastrukturer och algoritmer

Lab 2: Hitta dubbletter

Filip Strömbäck

1 Bakgrund till problemet

För att träna neurala nätverk behövs (oftast) en stor mängd data. Exempelvis sammanställde Nilsback och Zisserman¹ en stor samling bilder av blommor för att kunna träna ett neuralt nätverk att klassificera blommor. I just detta fallet består datamängden av 8189 jpg-filer i en mapp.

Ett problem med stora datamängder av den här typen är att det är svårt att få en bra översikt över vad datan innehåller. Exempelvis är det svårt att säkerställa att alla bilder är unika och att det inte finns dubletter. Det är viktigt att känna till om det finns dubletter inom maskininlärning eftersom det kan påverka både inlärningsprocessen och verifieringsprocessen. Finns exempelvis samma bild i både träningsdatan och testdatan så kommer det neurala nätverket med mycket stor sannolikhet svara rätt på bilden (den har ju sett "facit"), vilket kan ge en missvisande bild om hur bra nätverket presterar.

Detta problem är inte bara intressant inom maskininlärning. Det finns många andra situationer där det är intressant att hitta dubletter. Antag exempelvis att du vill arkivera bilder från Internet (likt the Internet Archive). För att spara på lagringsutrymme är det då intressant att identifiera dubletter, så att du bara behöver lagra en kopia av varje bild. Ett annat exempel är att söka igenom sin (osorterade) fotosamling efter foton som är nära identiska. I detta fall kanske det inte är intressant att ta bort dubletter, utan snarare att hitta alla liknande bilder så att du som användare kan välja den bästa.

2 Uppgiften

Laborationen består av två delar: 2a och 2b. Den första delen (2a) går ut på att implementera en hashtabell. Den andra delen (2b) går ut på att implementera ett program som kan undersöka alla bildfiler i en katalog och hitta bilder som är "lika". För att göra detta effektivt kommer vi att behöva hashtabellen från första delen.

Resten av instruktionen är uppdelad i två delar. En för lab 2a och en för lab 2b. Lämpligtvis gör du först 2a och sedan 2b. Om du kör fast på 2a går det däremot att börja med (och göra klart) 2b ändå. Mer information om det finns i instruktionerna för 2b.

När du är klar med båda delarna ska du svara på frågorna i filen `readme.txt` i de givna filerna innan du redovisar för assistent.

¹<https://www.robots.ox.ac.uk/~vgg/data/flowers/102/>

3 Lab 2a: Hashtabell

Målet med den här delen av labben är att implementera en hashtabell som är snabbare än den hashtabell som finns i standardbiblioteket (`std::unordered_map`), åtminstone för vissa användningsområden.

Målet ovan kan till en början låta svårt att åstadkomma. Standardbiblioteket är i allmänhet snabbt och robust. Därför är det sällan en bra idé att implementera saker som redan finns i standardbiblioteket själv, åtminstone om man inte har en bra anledning.² Resultatet blir ofta antingen långsammare eller mindre robust i slutändan.

Varför går då den här delen av labben ut på att gå emot rådet ovan? Det finns två anledningar. (1) Att implementera en hashtabell (eller andra datastrukturer för den delen) är ett mycket bra sätt att förstå hur de fungerar. Att implementera något kräver i regel att man har förstått alla detaljer. Det är alltså väldigt vanligt att man inser att det är en hel del detaljer man har missat, även om man har läst och gjort övningarna i OpenDSA innan. Denna förståelse är i sin tur central för att resonera om hur program som använder hashtabeller beter sig, och i vilka situationer hashtabeller är lämpliga att använda.

Den andra anledningen till att vi implementerar just en hashtabell är: (2) C++-standarden ställer vissa krav på `std::unordered_map` som gör att den i princip måste implementeras som en länkad struktur (kallas för *open hashing* eller *separate chaining* i OpenDSA). Tyvärr är länkade strukturer sällan optimala på moderna CPU:er i och med att de ofta orsakar icke-sekventiella minnesaccesser. Som tur är gör detta vårt arbete enklare. Om vi implementerar en hashtabell med exempelvis öppen adressering (kallas för *closed hashing* eller *open addressing* i OpenDSA) så är det ganska enkelt att slå standardbiblioteket!

Det är dock värt att notera att `std::unordered_map` inte är dålig på något sätt. Trots att den måste vara en länkad struktur så är den förhållandevis snabb. Den duger alltså bra i de allra flesta situationer. I och med att `std::unordered_map` är tillräckligt snabb och väldigt vältestad är det en bra idé att börja med den, och att bara leta efter alternativ om man får uppenbara prestandaproblem på grund av hashtabellen.³ Den hashtabell vi implementerar i den här labben kommer exempelvis inte ha ett helt komplett gränssnitt, så även om den duger till 2b så kanske den inte passar för alla program trots att den är snabb.

3.1 Givna filer (lab 2a)

Följande filer hör till den här delen av labben:

hash_map.h Innehåller malltypen `Hash_Map` som ska bli vår hashtabell. Mycket av det publika gränssnittet som behövs för labben är redan färdigt. Det som saknas är implementationen av de privata funktionerna som innehåller en `TODO`-kommentar.

hash_test.cpp Innehåller ett program som låter dig interaktivt testa din hashtabell, liknande programmet till lab 1.

hash_info.h Innehåller funktionen `debug_print` som skriver ut hur både en `Hash_Map` och en `std::unordered_map` ser ut. Du behöver inte ändra i eller läsa koden i filen för att lösa labben.

²Om du exempelvis vet något om din data som standardbiblioteket inte vet så är det ibland möjligt att göra något mycket bättre än standardbiblioteket.

³Det finns exempelvis en uppsjö mer eller mindre vältestade alternativa implementationer som bibliotek på internet.

3.2 Krav (lab 2a)

Filen `hash_map.h` innehåller malltypen `Hash_Map`. Implementationen är komplett så när som på de tre funktionerna `insert_key`, `find_key` och `erase_key`. Målet med labben är att implementera dessa funktioner så att hashtabellen fungerar korrekt och uppfyller kraven nedan. Du kan använda `hash_key` för att beräkna hashvärdet av en nyckel. Mer information om implementationen finns i avsnitt 3.3.

- Hashtabellen ska använda **öppen adressering** (*open addressing*) och **linjär sondering** (*linear probing*) för att hantera kollisioner.
- Funktionen `erase_key` ska **ta bort** element från hashtabellen. Borttagna element ska alltså **inte** ersättas med *tombstones*.
- Implementationen ska fungera för godtyckliga datatyper som har:
 - En default-konstruktor (dvs. en konstruktor som inte tar emot några parametrar)
 - En tilldelningsoperator (=)
 - Om typen används som nyckel behövs också en likhetsoperator (==), och en överlagring av `std::hash` (se avsnitt 3.4)

Exempel på datatyper som uppfyller dessa krav är `int`, `std::string`, `float`, etc.

- Implementationen ska i slutändan vara snabbare än `std::unordered_map` när du kör testet i den givna koden (alternativ 9 i testprogrammet).

Det är också värt att notera att följande saker *inte* behöver lösas:

- Du behöver *inte* tänka på att manuellt konstruera och destruera objekt i hashtabellen. Det innebär att vi kommer skapa fler instanser av nycklar och värden än vad som sätts in i hashtabellen. Det är därför vi behöver en default-konstruktor.
- Vi implementerar *inte* hela gränssnittet som `std::unordered_map` har. Exempelvis har vi bara `const_iterator`. En annan detalj är att det inte går att skriva `iter->first`, utan man behöver skriva `(*iter).first` i stället. `for (auto &&x : map)` fungerar dock som vanligt.

3.3 Implementation (lab 2a)

Konceptuellt lagrar hashtabellen en samling av par (nyckel $\rightarrow$ värde). Vi skulle kunna lagra dessa som en array av `std::pair<Key, Value>`. Det är dock inte optimalt i termer av cache-lokalitet. Gör vi det så kommer nyckeln för element 0 ligga bredvid värdet för element 0 och så vidare. När vi letar efter rätt nyckel i hashtabellen så behöver vi ofta jämföra flera nycklar för att hitta rätt element (vi kommer få kollisioner, så vi behöver ofta undersöka åtminstone ett fåtal). Då måste vi hela tiden "hoppa över" värdena för att komma till nästa nyckel. Problemet med det är att CPU-cachen arbetar med större block av minne, ofta större än ett helt nyckel-värde-par. På grund av det så kommer CPU-cachen spendera både tid och utrymme att läsa in värdena, trots att vi kommer hoppa över dem.

För att lösa detta problem så lagrar hashtabellen data i separata arrayer.⁴ Arrayen `keys` lagrar alla nycklar och arrayen `values` lagrar alla värden. Arrayerna är lika stora, så om vi har hittat rätt nyckel på index 5 i `keys`-arrayen så vet vi att motsvarande värde finns på index 5 i `values`-arrayen. Fördelen med detta är att alla nycklar ligger bredvid varandra i minnet, så vi behöver inte "hoppa över" något när vi letar efter rätt nyckel i `keys`-arrayen. Detta gör att vi utnyttjar cachen bättre.

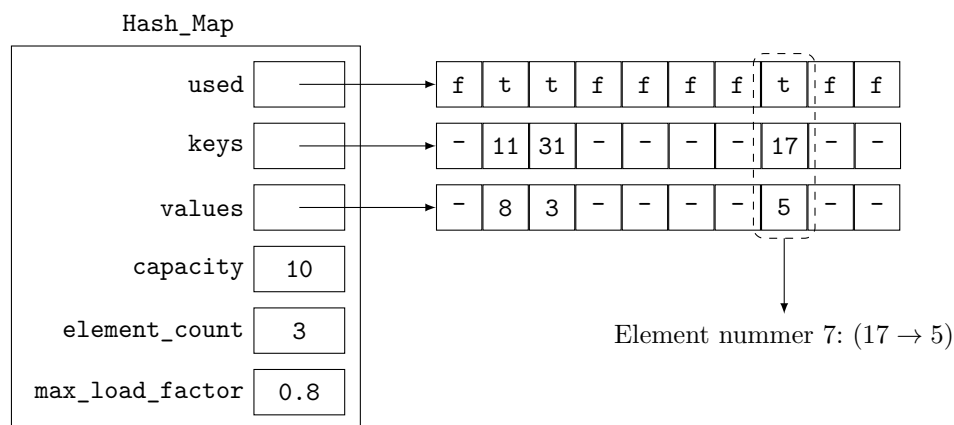
Det finns också en array som heter `used`. Den används för att komma ihåg vilka element i `keys` och `values` som innehåller faktiska nycklar och värden. När vi skapar arrayerna `keys` och `values` så kommer alla element innehålla *något* värde. Men eftersom vi inte vet vad `Key` och `Value` är för typer, eller vad för data som användaren av vår hashtabell sätter in, så kan vi inte avgöra om ett visst element är "riktig" data genom att bara titta på elementet.

⁴Detta kallas ibland för *structure of arrays*, medan `std::pair<Key, Value>` ibland kallas för *array of structures*.

Till exempel, om vi har en hashtabell där nycklarna är heltal (dvs. `Key` är typen `int`) så kommer alla element i arrayen `keys` ha något heltalsvärde från början.⁵ Vi skulle kunna börja med att sätta alla värden till exempelvis -100 för att indikera att de är lediga. Men vad händer då om den som använder hashtabellen sätter in nyckeln -100? Då kommer hashtabellen inte att fungera som den ska. Detsamma gäller för alla heltal som kan lagras i en `int`. Motsvarande gäller också för andra typer. Det finns alltså inte något värde vi kan använda för att indikera att ett element är "tomt". Det är därför vi behöver `used`.

Till sist är det värt att notera att `used`, `keys` och `values` är implementerade som C-arrayer (inte helt olik hur ex.vis `std::vector` fungerar internt). Det finns inget sätt att kontrollera hur stor en C-array är, så vi behöver hålla koll på längden själva. Vi lagrar längden av alla tre arrayer i variabeln `capacity`, eftersom längden av arrayerna motsvarar antalet element vi *kan* lagra som mest. Vi har också en variabel `element_count` som lagrar hur många av dessa element som vi faktiskt använder just nu. Variabeln `element_count` innehåller alltså antalet `true` i arrayen `used`. Variabeln `max_load_factor` anger hur full tabellen får bli innan vi ökar storleken på tabellen (dvs. `element_count` delat med `capacity` får inte överstiga `max_load_factor`).

Datarepresentationen sammanfattas i figuren nedanför:



Funktionerna som behöver implementeras är följande:

`size_t insert_key(const Key &key, const Value &value)`

Tar emot en nyckel och ett värde som ska sättas in i hashtabellen. Funktionen ska returnera det index som elementet sattes in på. Om nyckeln redan finns i hashtabellen så ska inget sättas in. Funktionen ska då returnera `capacity` (eller större) för att indikera att den inte satte in elementet. Funktionen ansvarar också för att omallokera arrayerna om de börjar bli fulla. Det löses dock med anropet till `grow_if_needed` som redan finns i koden.

`size_t find_key(const Key &key)`

Letar reda på nyckeln `key` i hashtabellen. Om den finns ska funktionen returnera det index som elementet finns på. Annars ska `capacity` (eller större) returneras.

`bool erase_key(const Key &key)`

Ta bort nyckeln `key` och motsvarande värde från hashtabellen. Om nyckeln togs bort ska `true` returneras, annars returneras `false`. Notera att elementet ska tas bort från tabellen. Implementeringen ska *inte* använda *tombstones*.

Tips: kom ihåg att du kanske behöver flytta på element som ligger efter det borttagna elementet i arrayen (både element med samma hashvärde och med andra hashvärden). Det är inte trivialt att få till korrekt, så det är värt att spendera lite tid på att fundera på vad som kan hända. Ett tips är att utnyttja `insert_key`. Man kan helt enkelt ta bort alla efterföljande element (ett åt gången) och sätta in dem igen.

Notera att `insert_key` och `find_key` returnerar index till element. De behöver dock också möjligheten att returnera något som inte är ett index: `insert_key` behöver kunna säga "insättning misslyckades

⁵Ofta 0 i testprogrammet, men det är inte garanterat.

eftersom nyckeln redan fanns”, och `find_key` behöver kunna säga ”nyckeln fanns inte”. Man skulle kunna tänka sig att använda exempelvis `-1` för detta. Det är dock problematiskt eftersom att `size_t` inte kan lagra negativa tal.

För att hitta andra alternativ så vill vi först förstå varför vi föreslog `-1` till att börja med. Anledningen är helt enkelt att `-1` uppenbart inte är ett giltigt index i arrayerna i hashtabellen. Målet är alltså att hitta ett annat värde som inte är ett giltigt index! Som tur är finns många sådana värden. Om vi exempelvis har 10 element i arrayerna så är alla värden större än eller lika med 10 ogiltiga. Det är därför vi säger att `insert_key` och `find_key` returnerar ett värde större än eller lika med `capacity` om de misslyckas. Dessa värden motsvarar ogiltiga index, och vi kan därför använda dem för att indikera fel!

3.4 Hashning (lab 2a)

Utifrån namnet *hashtabell* kan man enkelt inse att det är viktigt att kunna *hasha* nycklar. Detta görs av en *hashfunktion*. I mycket av den teoretiska litteraturen om hashtabeller (exempelvis OpenDSA) beskrivs hashfunktioner som $h(x, t)$, där x är nyckeln och t är hashtabellens storlek (dvs. kapaciteten eller antal element i arrayerna `keys`, `values` och `used`). Hashfunktionen ska då producera ett värde $0 \leq h(x, t) < t$, som kan användas som index i tabellen.

Fördelen med detta är att $h(x, t)$ innehåller all information vi behöver för att kunna analysera beteendet hos hashtabellen. Däremot är det inte lika smidigt implementationsmässigt. Om vi skickar med tabellstorleken så måste exempelvis alla hashfunktioner innehålla logiken för att ta hänsyn till tabellstorleken. Det gör också att vi inte kan använda hashtabellen till andra saker än hashtabeller.

En vanlig lösning på det problemet är att strukturera hashfunktionen som $h(x, t) = h'(x) \bmod t$. Man kan se det som att vi delar upp $h(x, t)$ i två delar. Först beräknar $h'(x)$ ett hashvärde av x . Notera att $h'(x)$ inte behöver känna till tabellens storlek. När det är gjort så tar vi hashvärdet och anpassar det till tabellens storlek med $\bmod$. Detta steg brukar göras internt av hashtabellen eftersom det kräver kännedom om hur hashtabellen fungerar internt (dvs. storleken av arrayerna inuti tabellen). I och med denna struktur så är det $h'(x)$ som man brukar kalla för *hashfunktion* när man pratar om implementationer i programspråk.

I C++ så implementeras $h'(x)$ av `std::hash`. Man kan se `std::hash` som en funktion med följande signatur: `size_t hash(const Key &value)`. Funktionen tar alltså emot ett värde av typen `Key` och ska producera en `size_t` som resultat. På moderna system så är `size_t` 64 bitar stor, så hashfunktionen returnerar heltal mellan 0 och $2^{64} - 1$. Man kan se hashfunktionen som att den tar den information som finns i datatypen `Key` och ”sammanfattar” informationen i de 64 bitar data som finns i `size_t`.

Det är värt att notera att `std::hash` inte är en vanlig funktion, så man kan inte skriva `std::hash(5)` och få ett hashvärde.⁶ Detta görs redan i den givna koden. Funktionen `hash_key` motsvarar $h(x, t)$. Likt ovan så börjar `hash_key` med att anropa `std::hash`. Den tar sedan hashvärdet och anpassar det till tabellens storlek med modulo. Du kan alltså använda `hash_key` för att hasha nycklar i din implementation.

Notera: hashfunktionen för heltal är identitetsfunktionen i GCC:s standardbibliotek. För heltal gäller alltså $h'(x) = x$. Det är trevligt, eftersom `hash_key` då blir $h(x, t) = x \bmod t$, vilket är precis vad som används i exemplen i OpenDSA. Sätter du tabellstorleken till 10 blir det också väldigt enkelt att beräkna hashfunktionen manuellt!

3.5 Testning (lab 2a)

Det är en bra idé att börja med små testfall som du kör interaktivt i testprogrammet. För att förenkla testning av lite större testfall kan det vara en bra idé att modifiera testprogrammet så att det lägger in några element direkt vid uppstart. Du kan också använda verktyget Testlab på kursshemsidan både för att hitta intressanta testfall, men också som hjälp att köra dina testfall. Testlab antar att både nyckel och värde är `int`. Ett tips är att testa med en tabellstorlek av 10, eftersom det då är enkelt att se vad hashvärdet av alla element kommer att vara.

⁶`std::hash` är en klassmall. `std::hash<T>` är alltså en klass. För att anropa hashfunktionen måste man först skapa en instans av klassen: `std::hash<int> h; h(10);`

När du tror att din implementation är korrekt kan du också använda det benchmark som finns i testprogrammet (alternativ 9). I och med att det sätter in och tar bort 1 miljon element så fungerar det som ett bra komplement till manuell testning och testning med testlab.

4 Lab 2b: Hitta dubbletter

Målet med den här delen av laborationen är att implementera ett program som undersöker alla bilder i en given katalog. Programmet ska sedan skriva ut (och visa) alla grupper av bilder som är "lika".

Exempelvis finns 7 bildfiler i mappen `tiny/` i de givna filerna. Bilderna `a1.jpg` och `a2.jpg` är nästan identiska. Detsamma gäller för bilderna `b1.jpg`, `b2.jpg` och `b3.jpg`. Resterande bilder är unika. Programmet ska därmed rapportera just detta: att bilderna `a1.jpg` och `a2.jpg` är "lika" varandra, och att de tre bilderna `b1.jpg`, `b2.jpg` och `b3.jpg` är "lika" varandra.

Vid första anblick verkar detta väldigt enkelt. Dock uppkommer snabbt två viktiga frågor:

1. Vi är egentligen inte intresserade av exakta dubbletter (därav att "lika" är inom citationstecken). Vi vill betrakta två bilder som "lika" även om det finns små skillnader (exempelvis komprimeringsartefakter, lite olika ljusstyrka, etc.). Detta gör att vi inte bara kan jämföra bilderna pixel för pixel, utan vi måste hitta ett sätt att bestämma om två bilder är "tillräckligt lika".
2. Om vi har ett stort antal bilder (exempelvis 8189 bilder på blommor) så kan vi inte jämföra alla par av bilder. Det skulle ta för lång tid. Vi behöver alltså hitta en bättre metod!

En lösning på problem nummer 1 finns implementerad i den givna koden. Det visar sig dock att denna lösning kräver att man jämför alla par av bilder. Den är därför inte lämplig för att även kunna lösa problem nummer 2. Uppgiften är alltså att hitta en lösning som löser båda problemen ovan samtidigt. En idé på hur detta kan göras finns beskriven i avsnitt 4.4.

4.1 Givna filer (lab 2b)

Till labben hör ett antal givna filer. Nedan finns en kort beskrivning av de filer som är intressanta att studera närmare i denna del (utöver hashtabellen från förra delen):

slow.cpp Innehåller en långsam implementation av ett program som hittar duplicerade bilder. Ditt program (**fast.cpp**) ska bli snabbare än detta program.

fast.cpp Innehåller ett skelett till ett program som hittar duplicerade bilder. Tanken är att detta program ska vara mycket snabbare (ca 10 gånger snabbare) än programmet i **slow.cpp** när antalet bilder blir stort. Notera dock att **fast.cpp** inte kommer att ge exakt samma utdata som **slow.cpp** i alla fall. Se *in-* och *utdata* för mer information.

image.h Innehåller en klass som representerar en bild. En bild kan som bekant betraktas som en 2-dimensionell array av pixlar. Klassen innehåller också grundläggande operationer för att manipulera bilder. Exempelvis finns en funktion **shrink** som skalar ner en bild till en lägre upplösning. Det går att lösa labben utan att förstå implementationen i **image.cpp**.

pixel.h Innehåller en klass som representerar enskilda pixlar i en bild. Pixelns färg är representerad som tre flyttal mellan 0 och 1. En för var och en av grundfärgerna röd, grön och blå. Klassen innehåller även funktionen **brightness()** som beräknar ljusstyrkan hos en pixel, samt operationer för grundläggande aritmetik.

load.h Innehåller hjälpfunktioner för att hantera filer. Här finns funktionen **list_files** som hittar alla filer av en viss typ i en katalog. Här finns också funktionen **load_image** som läser in en bildfil. Det går att lösa labben utan att förstå implementationen i **load.cpp**.

window.h Innehåller en klass som representerar ett fönster som används till att visa text och bilder på skärmen. Detta används för att rapportera vilka bilder som var "lika", men det kan också användas för att visa bilder och meddelanden för felsökning.

tiny/ Innehåller 7 bilder för felsökning.

small/ Innehåller ca 70 bilder för testning.

medium/ Innehåller ca 360 bilder för testning.

`/courses/TDDI16/lab2/large/` Innehåller alla 8189 bilder från Nilsback och Zisserman. Totalt är dessa ca 400 MB, så de följer inte med i de givna filerna. Arbetar du på skolans system finns de i den delade mappen `/courses/TDDI16/lab2/large/`, så du behöver inte kopiera dem till din hemkatalog om du inte vill. Arbetar du på en egen dator så finns filerna att ladda ner ifrån följande länk: https://www.ida.liu.se/~TDDI16/common/lab2_large.zip

4.2 In- och utdata (lab 2b)

Koden kan kompileras med kommandot `make`. Detta kompilerar båda programmen `slow.cpp` och `fast.cpp`. Vill du endast kompilera ett av dem kan du använda `make slow` eller `make fast` respektive. Första gången kan flaggan `-j` användas för att kompileringen ska gå snabbare (då startas flera kompileringsprocesser parallellt där det är möjligt).

Programmen kan sedan köras på vanligt vis (dvs. `./slow` eller `./fast`). Den katalog som ska genom-sökas anges som parameter på kommandoraden. Så för att använda mappen `tiny` kan man alltså köra exempelvis: `./slow tiny`. Om man vill söka genom den stora datamängden kan man även specificera en absolut sökväg, likt `./slow /courses/TDDI16/lab2/large`. På så vis behöver man inte kopiera bilderna till sin hemkatalog.

De båda programmen ska sedan rapportera alla duplicerade bilder genom att anropa `window->report_match()`. Funktionen tar en `std::vector<std::string>` som parameter. Denna `vector` förväntas innehålla sökvägen till två eller flera filer som är "lika". Sökvägarna skrivs både ut i terminalen och visas i fönstret som programmet skapar.

När `report_match` anropas så pausas körningen tills du har klickat i fönstret som visar bilderna. Detta för att du ska kunna se resultatet från körningen ordentligt. Om du inte vill att programmet väntar kan du ange flaggan `--nopause` på kommandoraden. Om du inte ens vill se fönstret kan du i stället ange flaggan `--nowindow`. Exempelvis: `./slow --nowindow small`

4.3 Testdata (lab 2b)

De givna filerna innehåller tre olika datamängder som är lämpliga att använda i olika stadier. Filerna i katalogerna `tiny/`, `small/` och `medium/` är namngivna för att enklare se vilka bilder som borde vara "lika" varandra. Filer med namn som börjar med `x` är bilder som ska vara unika. Resterande filer är "lika" alla andra filer som börjar med samma bokstav. Exempelvis är filerna `a1.jpg` och `a2.jpg` "lika". Detsamma gäller filerna `b1.jpg`, `b2.jpg` och `b3.jpg`.

Den lösning som föreslås nedanför kommer att hitta precis dessa dubbletter om den är korrekt implementerad. Ett korrekt program bör alltså skriva ut följande rader om man kör `./fast tiny`. Kodskelettet skriver ut lite ytterligare ledtext. Ordningen spelar ingen roll.

```
MATCH: tiny/a1.jpg, tiny/a2.jpg
MATCH: tiny/b1.jpg, tiny/b2.jpg, tiny/b3.jpg
```

Notera: eftersom `slow` använder ett annat sätt att jämföra bilder så kommer den inte alltid hitta samma dubbletter som din lösning av `fast`. Du kan alltså *inte* jämföra utdata mellan `fast` och `slow`. De kommer inte hitta samma dubbletter på alla testfall. Använd i stället namngivningen av filerna i `tiny`, `small` och `medium` enligt vad som anges ovan. Den motsvarar de dubbletter som `fast` borde hitta.

4.4 Lösningssidé (lab 2b)

Som nämndes i avsnitt 4 så finns två större problem som måste lösas för att kunna bygga en effektiv lösning på problemet. Det första är att vi egentligen inte är intresserade av att hitta bilder som är exakt lika, utan vi nöjer oss med bilder som är ungefär lika. Det andra är att vi inte vill behöva jämföra alla par av bilder, eftersom det tar lång tid.

Programmet `slow.cpp` löser det första problemet. För att inte kräva att bilderna är exakt lika skalas först alla bilder ner till 32x32 pixlar (som en bieffekt gör detta att programmet använder mindre minne än om alla bilder behövde sparas i full upplösning i RAM), och ljusstyrkan hos bilderna normaliseras. Sedan undersöks alla par av bilder. För varje par av bilder itererar programmet igenom alla pixlar i bilderna.

För varje par av pixlar så beräknas skillnaden i ljusstyrka. Skillnaden kvadreras, och medelvärdet av de kvadrerade värdena beräknas. Om medelvärdet är tillräckligt litet så betraktas bilderna som lika. Processen kan sammanfattas med följande formel (x och y är koordinater, a_{xy} och b_{xy} är ljusstyrkan hos individuella pixlar i de två bilderna):

$$d = \sum_{x=0}^{31} \sum_{y=0}^{31} (a_{xy} - b_{xy})^2$$

Detta är inte ett lämpligt sätt att lösa problem nummer 2, eftersom den kräver att vi jämför bilderna parvis. Vi behöver alltså en annan idé till programmet `fast.cpp`.

För att inte behöva undersöka alla par av bilder vill vi i `fast.cpp` i stället beräkna en "sammanfattning" av varje bild (representerat i koden av typen `Image_Summary`). Idén är att denna sammanfattning ska innehålla precis lagom mycket information om bilden, så att om två bilder har samma sammanfattning så är bilderna ungefär lika. Detta gör att vi kan lagra de beräknade sammanfattningarna i den hashtabell vi implementerade i del a för att snabbt kunna hitta bilder som är ungefär lika.

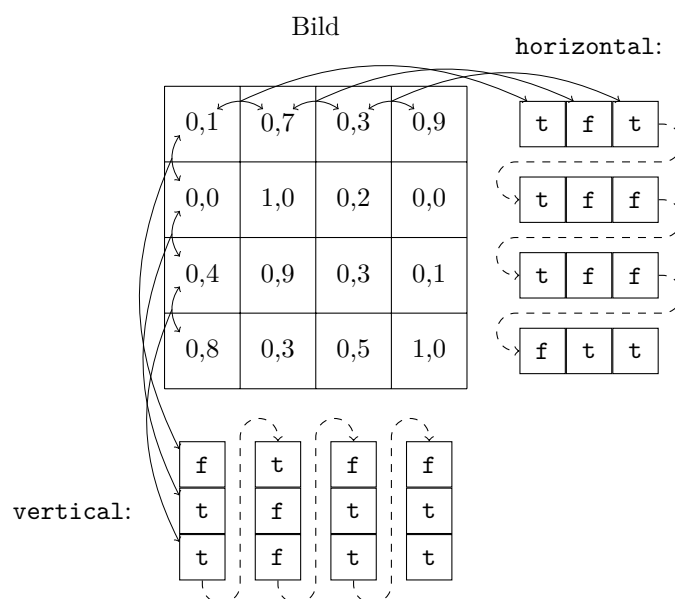
En stor fråga som kvarstår är då: hur beräknar vi en sådan sammanfattning av en bild? Sammanfattningen måste innehålla tillräckligt mycket information om bilden så att vi kan skilja på bilder som faktiskt är olika. Men den får samtidigt inte innehålla för mycket detaljer som vi som människor finner oviktiga. Misslyckas vi med att "kasta bort" dessa oviktiga detaljer så kommer vi att missa bilder som bara skiljer sig i små, oviktiga detaljer.

Ett sätt att göra detta på är enligt algoritmen nedan. Idén är att först skala ner bilden för att eliminera onödiga detaljer. Sedan undersöker vi förändringar i ljusstyrkan mellan pixlar i alla rader och kolumner i den nedskalade bilden. Ett sätt att se det på är att vi hittar horisontella och vertikala *konturer* i bilden. Mer konkret kan detta implementeras enligt nedan:

- Välj ett tal, `summary_size`. Valet `summary_size = 8` fungerar bra i detta fall, men testa gärna andra val.
- Skala ner bilden till en bredd och höjd av `summary_size + 1` med hjälp av medlemmen `shrink`. Alltså 9x9 om `summary_size` är 8.
- Iterera genom alla rader av pixlar. För varje par av pixlar, spara en 1:a (`true`) i `horizontal` om den högra är ljusare än den vänstra. Spara annars en 0:a (`false`) i `horizontal`.
- Iterera genom alla kolumner av pixlar. För varje par av pixlar, spara en 1:a (`true`) i `vertical` om den nedre är ljusare än den övre. Spara annars en 0:a (`false`) i `vertical`.

Processen illustreras med ett exempel i fig. 1. I exemplet är `summary_size = 3`, vilket innebär att bilden först skalas ner till 4x4 pixlar. Innehållet i `horizontal` beräknas sedan genom att iterera genom alla rader i bilden. Första elementet är `true` eftersom ljusstyrkan i pixel (1, 0) är högre än i pixel (0, 0) (dvs. pixlarna representerar att det blir ljusare åt höger). Nästa element är `false` eftersom pixel (2, 0) är mörkare än pixel (1, 0) (dvs. det blir mörkare åt höger). Detta görs för resterande par av pixlar i resterande rader, så att `horizontal` till slut innehåller 12 element.

Innehållet i `vertical` beräknas på liknande sätt, men denna gång itererar vi igenom alla kolumner i stället. Första elementet är `false` eftersom pixel (1, 0) är mörkare än pixel (0, 0) (dvs. det blir mörkare nedåt). Nästa element är `true` eftersom pixel (2, 0) är ljusare än pixel (1, 0) (dvs. det blir ljusare nedåt). På samma sätt görs detta för resterande par, så att `vertical` innehåller 12 element.



Figur 1: Illustration över hur en nedskalad bild kan "sammanfattas". Bilden här är 4x4 pixlar stor. Varje cell innehåller ljusstyrkan av pixeln. I **horizontal** och **vertical** indikeras värdet **true** med **t** och **false** med **f**.

4.5 Använda Image_Summary i hashtabeller

Vi behöver kunna använda vår `Image_Summary` som en nyckel i vår hashtabell. Som vi såg i del a så kräver detta att vår `Image_Summary` har en hashfunktion och en likhetsoperator. Från början har `Image_Summary` ingen av dessa, så vi måste implementera dem.

Värt att notera är att detta inte är unikt för `Hash_Map`. Hashtabellerna som finns i standardbiblioteket (`std::unordered_map`, `std::unordered_set`, etc.) ställer samma krav på `Image_Summary`, och fungerar på samma sätt i den här bemärkelsen. Implementerar vi en hashfunktion och en likhetsoperator för `Image_Summary` enligt nedan så kommer den alltså även att fungera med standardbibliotekets hashtabeller.

Likhetsoperatoren implementeras som vanligt, genom att definiera en `operator ==` på lämpligt ställe i koden (antingen som en medlem eller som en fri funktion). Att definiera en hashfunktion är tyvärr inte lika intuitivt. Man gör det genom att specialisera mallklassen `std::hash` enligt nedan.⁷

```
// Definiera en typ som specialiserar std::hash för vår typ:
template <>
class std::hash<Image_Summary> {
public:
    // Typen ska kunna användas som ett funktionsobjekt.
    // Vi behöver därför överlagra funktionsanropsoperatoren (operator ()).
    size_t operator ()(const Image_Summary &to_hash) const {
        // Beräkna hash här...
        return ?;
    }
};
```

Nu när vi vet hur vi kan definiera en hashfunktion är nästa steg att lista ut vad som behövs för att implementera en bra hashfunktion. En hashfunktion ska som bekant omvandla en instans av nyckeltypen (`Image_Summary` i detta fall) till ett heltal (`size_t` är ett 64-bitars heltal på de flesta moderna system). Resultatet från hashfunktionen kan alltså vara mellan 0 och $2^{64} - 1$.

⁷Hashtabellerna i C++ tillåter också att man skickar med ett funktionsobjekt till hashtabellen. Det är dock sällan smidigt i större program, gör man på detta sätt får hashtabellen en annan typ.

En hashfunktion måste uppfylla villkoret att om två indata a och b är lika ($a = b$) så måste de producera samma hash ($h(a) = h(b)$). Detta är inte svårt att uppfylla – det räcker att se till att funktionen inte använder globala variabler. En trivial (men väldigt dålig) hashfunktion kan exempelvis alltid returnera 0. Detta uppfyller villkoret ovan, och hashtabellen kommer därmed fungera korrekt.

Anledningen till att vi inte alltid vill returnera 0 är att trots att hashtabellen kommer att fungera så kommer prestandan att vara dålig. I och med att hashfunktionen alltid returnerar 0 så kommer hashtabellen inte att kunna skilja på olika nycklar, och det kommer därför se ut som att alla nycklar kolliderar med varandra. Att hantera kollisioner är dyrt, och de flesta operationer på hashtabellen kommer därmed ta linjär tid i stället för konstant tid.

För att utnyttja styrkan hos en hashtabell måste vi alltså ge den en bra hashfunktion. En bra hashfunktion försöker exponera så mycket information i nyckeln som möjligt i det beräknade hashvärdet. Detta så att om vi har två olika nycklar ($a \neq b$), så ska sannolikheten att de har olika hashvärden vara hög ($h(a) \neq h(b)$). Det gör såklart inget om ett fåtal kollisioner finns, men hashtabellen kommer att prestera bättre om vi kan hålla antalet kollisioner lågt.

En idé till hur man kan tänka när man bygger hashfunktioner kan illustreras med följande sekvens av hashfunktioner. Den första är sämst, och varje steg nedanför blir bättre och bättre:

- `return 0;`
- `return to_hash.horizontal[0];`
- `return 2*to_hash.horizontal[1] + to_hash.horizontal[0];`
- `return 4*to_hash.horizontal[2] + 2*to_hash.horizontal[1] + to_hash.horizontal[0];`
- ...

Mönstret är enklare att se om man tänker binärt. Idén ovan är att vi tänker oss att `to_hash.horizontal` innehåller binära siffror (0 och 1). Vi konverterar dem sedan till ett "vanligt" heltal genom att byta bas på talet med hjälp av multiplikation och addition (eller bitshift (`<<`) och eller (`|`) om det känns mer logiskt). Om man inser att 64 bitar inte kommer att räcka för utdata är en vanlig teknik att "blanda" två tal med hjälp av XOR-operatören (`x ^ y`).

4.6 Tips (lab 2b)

Nedan följer ett par tips för hur man kan bygga lösningen i mindre steg, och testa dessa steg på vägen så att det går att hitta eventuella buggar så tidigt som möjligt:

- Om du använder datamängden `tiny/` så kan du testa din implementation av `compute_summary` genom att för varje bild skriva ut innehållet i `Image_Summary` och manuellt jämföra utskriften för de 7 bilderna.
- För att testa koden som använder hashtabellen för att hitta dubletter innan man har implementerat en bra hashfunktion kan man tillfälligt använda en hashfunktion som alltid returnerar 0. Detta gör såklart att hashtabellen blir långsam, men i och med att hashtabellerna ger korrekta resultat ändå duger detta för att felsöka koden med mindre datamängder (exempelvis `tiny/` och `small/`).
- Om du misstänker att programmet inte fungerar på grund av din hashtabell i del a så kan du använda `std::unordered_map` i stället. All funktionalitet (förutom `buckets`) fungerar på samma sätt i både `Hash_Map` och `std::unordered_map`.
- Om din lösning är långsam kan det vara intressant att undersöka om din hashfunktion har många kollisioner. Detta går att göra exempelvis genom att helt enkelt skriva ut resultatet av hashfunktionen för alla bilder i `medium/` till standard output, och sedan hitta dubletter enklare genom att köra resultatet igenom kommandot `sort` (exempelvis: `./fast --nowindow medium | sort` om hashvärdet skrivs ut först på raderna). Då ser man enklare hur många anrop till hashfunktionen som ger samma resultat, och kan fundera på varför så är fallet.